

coefs

December 20, 2023

```
[ ]: from scipy.integrate import quad
import numpy as np
```

```
[ ]: ROUND_VALUE = 10
```

Objetivo: calcular los coeficientes de los filtros de paso bajo de las ondículas de Daubechies

Partimos g_k y queremos encontrar un m tal que $g(\xi) = |(m(\xi))|^2$ y m es filtro de paso bajo. $g_k(\xi) = 1 - c_k \cdot \int_0^\xi \sin(t)^{2k+1} dt$ con $\frac{1}{c_k} = \int_0^\pi \sin(t)^{2k+1} dt$

```
[ ]: def create_g_k(k):
    ck = 1/quad(lambda t: np.sin(t)**(2*k+1), 0, np.pi)[0]
    def g_k(w):
        return 1 - ck * quad(lambda t: np.sin(t)**(2*k+1), 0, w)[0]
    return g_k
```

La función $g_k(\xi)$ es un polinomio trigonométrico de orden $2k + 1$. Hay que hallar los coeficientes de este polinomio para representarlo como suma de exponenciales complejas.

$$g_k(\xi) = a_0 + a_1 \cdot \cos(\xi) + a_2 \cdot \cos(2\xi) + \dots + a_{2k+1} \cdot \cos((2k+1)\xi) \quad a_0 = \frac{1}{2\pi} \int_0^{2\pi} g(t) dt \quad a_k = \frac{1}{\pi} \int_0^{2\pi} g(t) \cdot \cos(kt) dt \text{ para } k > 0$$

Podemos escribir $g_k(\xi)$ como $g_k(\xi) = \sum_{n=-(2k+1)}^{2k+1} b_n \cdot e^{in\xi}$. $\cos(n\xi) = \frac{e^{in\xi} + e^{-in\xi}}{2}$ Por tanto $b_k = b_{-k} = \frac{a_k}{2}$ para $k > 0$

```
[ ]: def find_bk(g, k):
    '''
    Coeficientes para expresarlo como serie de exp(ikw).
    '''

    return quad(lambda t: g(t)*np.cos(k*t),
                0, 2*np.pi)[0] / (2*np.pi)
```

```
[ ]: def get_coefficients_gk(k):
    '''
    Calcula los coeficientes de la función g_k
    cuando se escribe como serie de exp(ikw)
    '''
```

```

'''

gk = create_g_k(k)

coefs = [0]*(2*(2*k+1)+1)
for i in range(2*k+1+1):
    cc = find_bk(gk, i)
    coefs[2*k+1+i] = cc
    coefs[2*k+1-i] = cc

return coefs

```

Con los coefs de g_k construimos el polinomio $P(z)$ que es de orden $2 * (2k + 1)$

```

[ ]: def get_roots_inside_unit_circle(coefs):
    '''
    Calcula las raíces dentro del círculo unitario
    '''

    roots = sorted(np.roots(coefs), key=abs)
    return roots[:len(roots)//2]

```

Hallamos las raíces de $P(z)$ y nos quedamos con las que están dentro del círculo unidad. Con estas raíces construimos el filtro m . Los coeficientes de $m(\xi)$ los calculamos a partir de sus raíces.

```

[ ]: def get_filter_coefficients(roots):
    '''
    Calcula los coeficientes del filtro a partir de las raices
     $h_0 = (-1)^n * r_1 * r_2 * \dots * r_n$ 
    ....
     $h_{(n-1)} = -(r_1 + r_2 + \dots + r_n)$ 
     $h_n = 1$ 
    '''

    coefs = [1]
    for r in roots:
        coefs = np.convolve(coefs, [1, -r])

    f = np.sqrt(2)/sum(coefs)

    #Normalizar
    return [f*c for c in coefs]

```

```

[ ]: def get_Daubechies_filter(k):
    coefs = get_coefficients_gk(k)
    roots = get_roots_inside_unit_circle(coefs)

```

```

filter_coefs = get_filter_coefficients(roots)

# Los coeficientes han de ser reales. Forzamos la parte imaginaria a 0
# para contrarestar errores de redondeo

filter_coefs = np.round(np.real(filter_coefs), ROUND_VALUE)
return filter_coefs

```

```

[ ]: # imprimimos valores de los filtros.
# (k+1 para seguir con la notación de la tabla que aparece en Daubechies Ten
↳ Lectures On Wavelets)
for k in range(1, 10):
    print(k+1, get_Daubechies_filter(k))

```

```

2 [ 0.48301272  0.83650296  0.22409406 -0.12939618]
3 [ 0.33372612  0.8073373  0.45832921 -0.13534541 -0.08494854  0.03511487]
4 [ 0.23468326  0.71952911  0.62455657 -0.03387585 -0.18402383  0.03185589
  0.03189078 -0.01040237]
5 [ 0.16857264  0.61844211  0.71538411  0.11676352 -0.23975351 -0.0237844
  0.07451958 -0.00748168 -0.01161604  0.00316723]
6 [ 0.12688726  0.52997279  0.74634431  0.26056628 -0.23875705 -0.10216618
  0.09743918  0.01789914 -0.02876226  0.00178173  0.00395536 -0.000947 ]
7 [ 9.46686468e-02  4.45370636e-01  7.42629221e-01  3.97224965e-01
 -1.89872109e-01 -1.85849149e-01  8.95578146e-02  6.04587859e-02
 -3.95039758e-02 -1.00608029e-02  1.09905336e-02 -3.27947800e-04
 -1.36335030e-03  2.90294100e-04]
8 [ 7.56183945e-02  3.85060439e-01  7.18945146e-01  4.89641445e-01
 -1.21021744e-01 -2.43088188e-01  5.70608150e-02  1.00010821e-01
 -3.55918657e-02 -2.87641990e-02  1.55919989e-02  4.25624250e-03
 -3.92645520e-03  7.47449000e-05  4.30507900e-04 -8.45409000e-05]
9 [ 5.78230850e-02  3.22257092e-01  6.78289029e-01  5.74982962e-01
 -2.18863002e-02 -2.85222058e-01 -2.12999150e-03  1.35301343e-01
 -1.35959896e-02 -5.20586723e-02  1.33546470e-02  1.37179644e-02
 -6.03158150e-03 -1.89602270e-03  1.42725760e-03 -1.63680000e-06
 -1.43369300e-04  2.58025000e-05]
10 [ 4.89996000e-02  2.85944402e-01  6.43265743e-01  6.15801787e-01
  5.48318035e-02 -2.95297876e-01 -5.47916537e-02  1.49628017e-01
  1.40932435e-02 -6.76963911e-02  4.30391430e-03  2.35684638e-02
 -5.41544260e-03 -5.50647690e-03  2.25281020e-03  6.71163200e-04
 -4.75749800e-04  9.21200000e-07  4.25467000e-05 -7.26190000e-06]

```